

Task: Garden

Proposed by: Normunds Vilcins

In this task, we would like to compute the number of possible paths that could have led each group to the specified intersection P , using the given number of steps K .

Notice that each path is completely determined by its initial intersection. Thus, to compute the number of possible paths, we only need to check whether each intersection, if used as an initial intersection, would bring the group to intersection P after exactly K steps. As we need to check all N initial intersections for each of the Q groups, an efficient algorithm for checking whether the group will be at intersection P after exactly K steps is needed, which will be discussed in following sections.

1 Graph construction

This problem can be treated as a graph problem. A natural approach is to construct a graph G containing the following information: for each intersection, where the group would move to. Since they may take only one of the two most beautiful trails, we will use two vertices to represent an intersection. Namely, for the i -th intersection, let v_i represent this intersection where the next chosen trail must be the most beautiful trail incident to it, and v'_i represent this same intersection but where the next chosen trail must be the *second* most beautiful trail incident to it (or the most beautiful trail if no alternative is available). In other words, v_i represents the i -th intersection when the last taken trail is *not* the most beautiful trail incident to this intersection, and v'_i represents this intersection when the last taken trail is the most beautiful one incident to this intersection.

Now for each vertex, we add an outgoing edge representing the most beautiful or second most beautiful trail, according to the conditions mentioned above. With this, G will contain $2N$ vertices, and exactly one outgoing edge from each vertex.

The construction of the graph G takes $O(M + N)$ running time by first creating $2N$ vertices, then scanning through the array R representing trails, and conditionally adding these edges to G under the described conditions.

2 An $O(M + NKQ)$ solution

A simple way to check where the couple would arrive after K steps is to simulate their path, for each intersection as an initial vertex. Since they always choose the most beautiful trail in the first steps, the corresponding starting vertices in G are $v_0, \dots, v_{N-1}$.

To simulate their walk, we simply start at some vertex v_s , then follow the unique outgoing edge for that vertex, and repeat this process for K steps. Since the vertices corresponding to

intersection P are v_P and v'_P , then this path ends at this intersection if and only if after K steps, we stop at one of these vertices. That is, to find the number of possible paths, we simulate their walk for all possible initial vertices v_i , and count the number of starting vertices that end at v_P or v'_P after K steps.

Clearly, this process takes $O(K)$ total running time for each starting vertex. Since there are N possible starting vertices and Q questions, this algorithm takes $O(M + NKQ)$ running time, including graph construction. This running time is sufficient to fully solve subtask 1.

3 An $O(M + NQ \log K)$ solution

As K becomes large in subtask 2, we need a better way to simulate the algorithm mentioned in the previous section. Notice that the edges in G represents 1-step traveling. To simulate faster, we will use the permutation-composition approach.

We first precompute the result of 2^k -step traveling from each vertex in G , where $k = 0, 1, 2, \dots$, using a technique similar to successive squaring. Let $T_{v,2^k}$ represents the vertex we arrive at after traveling from v for 2^k steps. Then for $k = 0, 1, 2, \dots$, we can compute $T_{v,2^k}$ easily: If $k = 0$, then the destination is specified in G ; otherwise, we compose the two paths of length 2^{k-1} using the formula $T_{v,2^k} = T_{T_{v,2^{k-1}},2^{k-1}}$. In other words, traveling 2^k steps from v is the same as traveling 2^{k-1} steps from v , then from that vertex, continue for 2^{k-1} more steps.

Then, notice that for each value of K , we can decompose this number into sum of distinct, non-negative powers of two. Suppose that $K = 2^{k_1} + 2^{k_2} + \dots + 2^{k_l}$ where $k_1 < k_2 < \dots < k_l$ for some positive integer l . Then the result of traveling K steps from v can be found by simply composing travelings of $2^{k_1}, 2^{k_2}, \dots, 2^{k_l}$ that we have precomputed. Using this technique, therefore, we can compute the destination for each starting intersection in $O(\log K)$ running time. Note that since $K < 2^{30}$, we only need to compute $T_{v,2^k}$ for $k = 0, 1, 2, \dots, 29$.

This algorithm takes $O(N \log K)$ extra running time to compute the values of $T_{v,2^k}$, as each of them can be computed in constant time. Then, we can find the destination of each path in $O(\log K)$. Thus, the total running time is $O(M + NQ \log K)$, which is sufficient to fully solve subtask 2.

4 An $O(M + NQ)$ solution

Let us consider a more general question of determining whether a path starting at vertex s with length K ends at vertex t . Recall that each vertex in G has exactly one outgoing edge. So, from any initial vertex, by simply following these edges, we will eventually enter a cycle. Thus, if we start at s , exactly one of the following conditions are met:

- We never reach t .
- We reach t just once exactly after some number of steps F . In this case, t is reachable, but is not on any cycle.
- We reach t for the first time after some number of steps F , and enter it every C steps. In this case, t is reachable, and is on a cycle of size C .

For our purpose of solving the problem, s can vary depending on our initial vertex; namely, it can be any of the vertices v_i for $i = 0, 1, \dots, N - 1$. However, t can only be vertices v_P and v'_P . Since t does not vary very much, it is easier to check whether t is on a cycle, and whether it is possible to reach t from s .

To solve this problem, we create the graph G^T , which is the same as graph G with its edges reversed. Then, we perform a depth-first search on this graph starting at t . During this search, we keep track of the distance of each reachable vertex from t . This number is the distance from s to t in G ; that is, the number of steps F that brings us from s to t for the first time. At the same time, if we reach some vertex with a departing edge to t , then we obtain the size of the cycle C , which is the distance from t to that vertex plus 1.

Thus, whether the path in G starting at s with length K ends at t can be determined as following:

- If we cannot reach s in G^T , then this path in G cannot end at t .
- If we reach s in G^T after F steps, but t is not on a cycle, then this path in G ends at t if and only if $K = F$.
- If we reach s in G^T after F steps, and t is in a cycle of size C , then this path in G ends at t if and only if $K = F + nC$ for some non-negative integer n .

For our task, the path starting at the i -th intersection with length K will end at intersection P if and only if the path in G starting at v_i with length K ends at v_P or v'_P . Note that during the implementation, we do not need to create graph G , but we can create G^T directly. It is also convenient to first create an array for storing F_s for each vertex s . We then initialize F_s and C to ∞ , and update them during the depth-first search. We perform the search twice, starting at v_P and v'_P , respectively.

Since the depth-first search takes $O(M + N)$ running time and each query can be checked in constant running time, this algorithm takes $O(M + NQ)$ running time in total, which is sufficient to obtain full marks for this task.

Task: Race

Proposed by: Martin Fixman

Given a tree T with N nodes, this task asks for a path P^* of length K with the minimum number of edges. It looks like a usual dynamic programming task. However, when K is large, another approach is required.

The model solution for this task follows a divide-and-conquer approach.

Consider a node u in the graph. There are two possible cases: when node u belongs to the solution path P^* ; or when node u does *not*.

In the second case, we can delete node u from the tree and break it into smaller trees. We can then recurse on each of the resulted trees to find the solution.

With this general approach in mind, we have to answer the following questions.

- How to find the best path that contains node u ?
- How to choose u to achieve a better running time?

Note that the second question is very important because if we can guarantee that the sizes of all resulting trees are small, we can bound the number of recursion levels.

1 Finding the solution containing u

Consider the case that P^* contains node u . Let's consider a simpler case where we only want to find if there exists a path of length exactly K that contains u .

If u is one of the endpoints in P^* , we can find the path using one application of depth first search (DFS).

However, if u is “inside” P^* , then two of u 's adjacent nodes x and y must also be in P^* . Thus, we shall find x and y .

Consider some node w adjacent to u . With one application of DFS, we can find the set L_w of all path lengths for all paths starting at u and containing edge (u, w) .

Hence, to find x and y , we need to find two nodes x and y such that there exists a pair $\ell_x \in L_x$ and $\ell_y \in L_y$ for which $\ell_x + \ell_y = K$. This can be done by DFS from u through every edge (u, w) for all adjacent nodes w with careful book keeping using an array $A[0, \dots, K]$ of size $K + 1$.

The running time for this step is $O(N)$.

2 Finding the right node

Our goal is to find node u such that after deleting u , each resulting tree is sufficiently “small.” In this case, we shall find node u such that each remaining tree has at most $N/2$ nodes. We shall refer to node u as the *central node*.

It is not clear if such a node exists. So let’s argue about that first.

Pick an arbitrary node v as a candidate. Denote by $T' = T \setminus \{v\}$ the forest obtained by deleting v from T . For each node w adjacent to v , denote by T_w the tree containing w in T' . If every tree $T_w \in T'$ has at most $N/2$ nodes, we are done and v is the required central node.

Otherwise there exists one tree T_w that contains more than $N/2$ nodes. (Note that there can be only one tree violating our criteria.) In this case, we pick w as our new candidate and repeat the process.

This process will eventually stop at some candidate node and that’s the required central node. To see this, note that after leaving v , we shall never go back to pick v again; since there are N nodes, the process can repeat at most N times.

After knowing that the central node exists, there are many ways to find it. We can follow the process directly as in the argument. But this is too slow to be useful.

The following are two procedures that find the central node in $O(N \log N)$ time and $O(N)$ time.

2.1 Bottom-up approach

We can find node u in a bottom-up fashion. We shall keep a priority queue Q of all “processed” subtrees using their sizes as weights.

We maintain, for each node, its state which can either be *new* or *processed*; initially all nodes are new. Every node also has a weight. Initially every node has weight of 1.

We start with all leaf nodes in Q . Note that each node in Q is every node which has all but one of its adjacent nodes processed. For each node $v \in Q$, we denote by $p(v)$ the unique neighbor of v which is new.

While there are nodes in Q , we extract node v with the smallest weight. We update v state to processed and increase the weight of $p(v)$ by v ’s weight. If all but one neighbor of $p(v)$ are processed, we insert v into Q .

Using this procedure, the last node inserted to Q is the desired central node.

2.2 DFS with bookkeeping

With DFS and a good bookkeeping, we can find the central node in $O(N)$.

We pick an arbitrary node r to start a DFS. With this procedure, we can consider T as rooted at r and the parent-child relationship between adjacent pairs of nodes are clearly defined. While performing DFS, we compute, for each node v , the number of its descendants $D(v)$.

With this information, we can figure out if a candidate u is the central node. For each node w adjacent to u , if w is one of u ’s children, the size of the resulting tree containing w after deleting

u is $D(w) + 1$. If w is u 's parent, the size of the resulting tree containing w after deleting u is

$$n - 1 - \sum_{v \in Ch(u)} (D(v) + 1),$$

where $Ch(u)$ are a set of children of u . If the size of each resulting tree is at most $N/2$, u is the desired central node. The time needed to check u is proportional to u 's degree. Therefore, we can check all nodes in time $O(N)$

3 Running time

Let $\mathcal{T}(N)$ be the worst-case running time when the tree has N nodes. We can write the recurrence as

$$\mathcal{T}(N) = A(N) + cN + \sum_i \mathcal{T}(N_i),$$

where $A(N)$ is the time for finding u , N_i is the size of the i -th new trees, and c is some constant.

Since we know that $N_i \leq N/2$, there are at most $O(\log N)$ levels of the recursion.

If we use $O(N)$ -time to find u , each level would run in time $O(N)$ and the total running time is $O(N \log N)$. If we use a slower $O(N \log N)$ -time procedure, the total running time will be $O(N \log^2 N)$.

4 Notes

There are other heuristics for finding u that *do not* always work. Here are some examples.

- In divide-and-conquer solution, the highest degree node is picked.
- In divide-and-conquer solution, the node that minimizes the maximum distance to any node is picked.

Task: Rice Hub

Proposed by: Christian Kauth

The key insight to solving this problem is the observation that for any K rice fields located at $r_0 \leq r_1 \leq \dots \leq r_{K-1}$, the transportation cost from all these K fields is minimized by placing the rice hub at a median. For example, when $K = 1$, the hub should be at r_0 , and when $K = 2$, placing it between r_0 and r_1 is optimal. In this problem, we will place the rice hub at $r_{\lfloor K/2 \rfloor}$ for simplicity. Following this observation, we denote a solution by a sequence $S \subseteq \langle r_0, \dots, r_{R-1} \rangle$ and let $|S|$ denote the length of S , which is the solution's value (the number of rice fields whose rice will be transported to the hub). The cost of S is $\text{cost}(S) = \sum_{r_j \in S} |r_j - h(S)|$, where $h(S)$ is the $\lfloor |S|/2 \rfloor$ -th element of S .

1 An $O(R^3)$ solution

Armed with this, we can solve the task by a guess-and-verify algorithm. We try all possible lengths of S (ranging between 1 and R). Next observe that in any optimal solution S^* , the rice fields involved must be contiguous; that is, S^* is necessarily $\langle r_s, r_{s+1}, \dots, r_t \rangle$ for some $0 \leq s \leq t \leq R - 1$. Therefore, there are $R - K + 1$ solutions of length K . For each choice of S , we compute $h(S)$ and the transportation cost in $O(|S|)$ time and check if it is within the budget B . This leads to an $O(R^3)$ algorithm, which suffices to solve subtask 2.

2 An $O(R^2)$ solution

To improve it to $O(R^2)$, we will speed up the computation of $\text{cost}(S)$. Notice that we are only dealing with consecutive rice fields. Thus, for each S , the cost $\text{cost}(S)$ can be computed in $O(1)$ after precomputing certain prefix sums. Specifically, let $T[i]$ be the sum of all coordinates to the left of rice field i , i.e., $T[0] = 0$ and $T[i] = \sum_{j=0}^{i-1} X[j]$. Then, if $S = \langle r_s, \dots, r_t \rangle$, $\text{cost}(S)$ is given by $(p - s)r_p - (T[p] - T[s]) + (T[t + 1] - T[p + 1]) - (t - p)r_p$, where $p = \lfloor (s + t)/2 \rfloor$. This $O(R^2)$ algorithm suffices to solve subtask 3.

3 An $O(R \log R)$ solution

Applying a binary search to find the right length in place of a linear search improves the running time to $O(R \log R)$ and suffices to solve all subtasks.

4 An $O(R)$ solution

We replace binary search with a variant of linear search carefully designed to take advantage of the feedback obtained each time we examine a combination of rice fields. In particular, imagine adding in the rice fields one by one. In iteration i , we add r_i and find (1) S_i^* , the best solution that uses only (a subsequence of) the first i rice fields (i.e., $S_i^* \subseteq \langle r_0, \dots, r_i - 1 \rangle$), and (2) S_i , the best solution that uses only (a subsequence of) the first i rice fields and *contains* $r_i - 1$. This can be computed inductively as follows. As a base case, when $i = 0$, both S_i and S_i^* are just $\langle r_0 \rangle$ and cost 0, which is within the budget $B \geq 0$. For the inductive case, assume that S_i^* and S_i are known. Now consider that S_{i+1} is S_i appended with r_i , denoted by $S_i \cdot r_i$, if the cost $\text{cost}(S_i \cdot r_i)$ is at most B , or otherwise it is the longest prefix of $S_i \cdot r_i$ that costs at most B . Furthermore, S_{i+1}^* is the better of S_i^* and S_{i+1} . To implement this, we represent each S_i by its starting point s and ending point t ; thus, each iteration involves incrementing t and possibly s , but s is always at most t . Since $\text{cost}(\langle r_s, \dots, r_t \rangle)$ takes $O(1)$ to compute, the running time of this algorithm is $O(R)$ and suffices to solve all subtasks.

Task: Crocodile

Proposed by: Mihai Pătraşcu

1 Subtask 1: Trees

In this subtask, we can infer that the underlying graph is a tree and the exit chambers are precisely the leaves. This is simpler than the general case but will be instructive for the next subtasks. To begin, let $L(u, v)$ denote the time to travel along the corridor connecting node u to node v . For simplicity, we will root the tree at the starting node (chamber 0).

The crucial observation then is that in any successful escape plan, the instruction at each node, if reachable from the root, will always tell Somying to move further away from the root (otherwise, she can be forced to cycle around the underground city forever). This leads to a simple dynamic programming (DP) solution: Let $T[u]$ denote the best time to reach an exit chamber. First, $T[u] = 0$ if u is a leaf. Otherwise, if u has children $v_1, \dots, v_k$, ordered such that $T[v_1] + L(u, v_1) \leq T[v_2] + L(u, v_2) \leq \dots \leq T[v_k] + L(u, v_k)$, then $T[u] = T[v_2] + L(u, v_2)$. The problem statement guarantees that $k \geq 2$.

It is easy to show inductively that $T[u]$ is indeed the best time starting at u . Specifically, we prove that first, Benjamas can reach an exit chamber from u in $T[u]$ time regardless of what the gatekeeper does, and second, the gatekeeper can force Benjamas to spend $T[u]$ time in the underground city. Clearly, if u is a leaf node and hence an exit chamber, $T[u]$ is 0. Assuming inductively that $T[v_i]$ is as claimed for each child v_i of u , we have that in time $T[v_i] + L(u, v_i)$, Benjamas can reach an exit from u , through (u, v_i) , if the corridor is not blocked. Since the evil crocodile can only block one corridor at a time, he can force Benjamas to spend $T[v_2] + L(u, v_2)$, by blocking (u, v_1) —blocking any other corridor only helps Benjamas reach an exit faster, in $T[v_1] + L(u, v_1)$ time.

To compute this DP, we traverse the tree in postorder (i.e., the leaves are visited first and each parent is visited after all its children); the final answer is stored in $T[0]$. The computation at each node involves finding the second smallest value, which can be done in $O(d_u)$ time. Here, d_u denotes the (out-)degree of u . Therefore, the total running time is $C \cdot \sum_u d_u = O(N)$, for some positive constant C , because the degrees of the tree nodes sum to $2(N - 1)$.

2 Subtasks 2 and 3: General Graphs

The challenge in generalizing our current algorithm to the general setting is the lack of a clear sense of direction; in our current algorithm, we know Benjamas must always move further from the root as necessitated by the tree structure.

A moment's thought reveals striking similarity between Dijkstra's single-source shortest path algorithm and our algorithm for trees. Indeed, the algorithm iteratively grows the frontier set,

where at any point in time, a node u is in the set if $T[u]$ has been determined. From this view, our algorithm for trees can be seen as running Dijkstra’s algorithm starting from the exit chambers. The algorithm is standard except for how the cost at a node is defined.

Consider the following algorithm: For all nodes u , set $T[u]$ to ∞ except when u is an exit chamber, set $T[u] = 0$. Initially, the frontier set S contains exactly the exit chambers. During the execution of the algorithm, we maintain that for $w \notin S$, the cost of w can be *conceptually* computed by producing the list¹ $\{v \in N(w) : T[v] + L(w, v)\}$, sorting this list, and returning the second value (i.e., the second smallest value in this list). When a node u enters the frontier (it has the lowest cost among non-frontier nodes), $T[u]$ is set to the cost of u at that moment.

Claim 1. *For each node u , Benjamas can reach an exit from u in $T[u]$ time regardless of what the gatekeeper does. Furthermore, the crocodile gatekeeper can force Benjamas to spend $T[u]$ time in the underground city.*

This claim can be shown by a similar inductive argument as in the tree case and by observing that as in Dijkstra’s algorithm, once a node enters the frontier its cost cannot decrease because the edges have positive cost.

Implementation Details. For each node, we can maintain its cost by keeping two numbers—the smallest value and the second smallest value—which can be updated in $O(1)$ when the neighboring values change. Using this, Dijkstra’s algorithm takes $O((M + N) \log N)$ using a heap or $O(N^2)$ without using one.

¹ $N(w)$ denotes the set of neighbors of w .

Task: Elephants

Proposed by: Mihai Pătraşcu

1 Subtask 1

In subtask 1, there are only two elephants. Thus we can easily determine the number of required cameras in constant time. Namely, we only need one camera if the elephants are at most distance L apart; otherwise, we need two cameras.

2 Subtasks 2 and 3

If elephants are at positions $x_0, x_1, \dots, x_{N-1}$, such that $x_i \leq x_{i+1}$ for all $0 \leq i < N - 1$, we can compute the minimum number of cameras required using a greedy algorithm. We start with an empty set of cameras. While the current set of cameras do not cover all elephants, we choose an elephant which is not already covered with the minimum position x_j , and place a camera to cover elephants at positions in $[x_j, x_j + L]$. We can implement this procedure to run in time $O(N)$ by iterating through the list of sorted positions once.

Each time an elephant moves in the show, we can update the sorted list of positions in $O(N)$ time. Hence, we have an $O(NM)$ solution to this problem, which is sufficient to fully solve subtask 2, and an adequate optimization is required to fully solve subtask 3. However, a faster algorithm is required for subtasks 4 and 5.

3 Subtasks 4 and 5

3.1 Bucketing elephants

To find the number of cameras, instead of iterating through all elephants, we shall build a data structure that allows us to “jump” over lots of elephants.

We maintain k buckets $B_0, B_1, \dots, B_{k-1}$ of elephants such that buckets with lower indices store elephants with lower positions, i.e., for any $0 \leq b < k - 1$, for all $x_i \in B_b$ and $x_j \in B_{b+1}$, $x_i \leq x_j$. Also, elephants in each bucket are sorted according to their positions.

The goal is to make sure that to find the number of required cameras, one needs to visit each bucket once. For simplicity, we will always place cameras so that the left-most position covered by a camera is the position of some elephant.

Consider bucket b with p elephants. Denote the list of indices of elephants in B_b as $e_0, e_1, \dots, e_{p-1}$ (that is, $x_{e_i} \leq x_{e_{i+1}}$). Given an elephant e_i , we would like to answer the following two questions quickly:

- **Q1:** If we would like to cover all elephants starting from e_i (i.e., elephants in the set $\{e_i, e_{i+1}, \dots, e_{p-1}\}$), how many cameras are needed?
- **Q2:** What is the highest position that these set of cameras cover?

For elephant e , denote the answer for Q1 for as $J(e)$ and the answer to Q2 as $T(e)$.

If we have these answers for every elephant in every bucket, we can find the number of cameras in time $O(k \log N)$ as follows.

We start by placing the camera at the first elephant in bucket B_0 , so that the position of this elephant is the left-most position covered by this camera. Now consider placing a camera at elephant e in bucket B_i in the same fashion. We know that to cover all elephants in B_i , we have to use $J(e)$ cameras and these cameras cover positions up to $T(e)$. We find the first elephant e' not covered by these cameras in the next bucket B_{i+1} by binary searching for the elephant in B_{i+1} whose position is minimum but greater than $T(e)$. Then, we start placing the camera at elephant e' in bucket B_{i+1} .

We repeat this step until we reach the last bucket. Since each step runs in $O(\log N)$ time (from binary search), we spend $O(k \log N)$ time as required.

We can precompute the answers for Q1 and Q2 for elephants in B_i in $O(|B_i|)$ time by iterating over each elephant e_j from e_{p-1} to e_0 and keeping a pointer to the first elephant outside the range $x_{e_j} + L$. For implementation details, please see the appendix.

It is crucial to note that we can process each bucket independent of all other buckets.

3.2 Updating the data structure

When an elephant e moves, we will have to update two buckets: the current bucket B_i and the new bucket B_j . This can be done in time proportional to the current size of the bucket. To find the current bucket of e we can store a pointer from e , but it takes $O(k)$ to find the new bucket anyway. Therefore, the running time for the update is $O(k + |B_i| + |B_j|)$.

Note that the time depends heavily on the size of each bucket. Initially, we would have about N/k elephants in each bucket, but the number may grow as elephants can move. To keep the size of each bucket bounded above by $O(N/k)$, we will rebuild the whole data structure for every $\lceil N/k \rceil$ updates. The rebuilding takes time $O(N)$.

3.3 Choosing the right parameter

We need to handle M updates and answer one question after each of these updates. The total running time is

$$O(M \cdot k \log N) + O(M \cdot (k + N/k)) + O(M \cdot (N/(N/k))),$$

where the first term denotes the total query time, the second term denotes the total updating time, and the last term denotes the total rebuilding time.

Choosing $k = \sqrt{N}$ gives the running time of $O(M\sqrt{N} \log N)$, which is sufficient to obtain full marks for this problem. However, an inefficient implementation may not be able to solve subtask 5. For example, using the set data structure in the C++ Standard Template Library can introduce an extra factor of $\log n$ to the running time of rebuilding the data structure. This can be avoided by using simple arrays.

A Processing each bucket

To simplify the presentation, we add a dummy elephant e_p at position $x_{e_{p-1}} + L + 1$. Also, we let $y_j = x_{e_j}$ be the position of the j -th left-most elephant in bucket B_i .

We consider each elephant j from the right-most elephant e_{p-1} to the left-most one. We also maintain an index t that points to the left-most elephant e_t whose position $y_t > y_j + L$. Initially, $j = p - 1$ and $t = p$.

For each elephant e_j , we will compute $J(e_j)$ and $last(e_j)$, the left-most elephant in the right-most camera in the set of cameras covering $\{e_j, e_{j+1}, \dots, e_p\}$.

For the dummy node, we let $J(e_p) = 0$ and $last(e_p) = e_p$. For elephant e_j , we check if we need to move t , i.e., if the position of e_{t-1} is greater than $y_j + L$; if that's the case we find the smallest t such that $y_t > y_j + L$. We let $J(e_j) = J(e_t) + 1$ and $last(e_j) = last(e_t)$.

Finally, for each elephant e_j such that $last(e_j)$ points to the dummy elephant e_p , we change $last(e_j)$ to e_j .

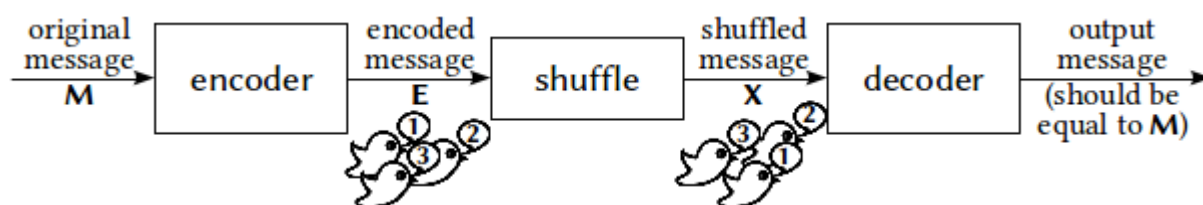
We can complete the process using only one pass over all elephants in the bucket B_i and note that the pointer t moves over each elephant only once. Thus, the running time is $O(|B_i|)$ as claimed.

To answer question Q2 for each elephant e_j , we report $y_{last(e_j)} + L$.

Task: Parrots

Proposed by: Jittat Fakcharoenphol

For this task, contestants are given a message M of length N , which consists of a sequence of integers between 0 and 255, inclusive, and they are asked to find an encoding and decoding scheme such that the encoded message X are invariant up to permutations. The encoded message should be as concise as possible. The scores will be rewarded based on the *ratio* of the length of the encoded message X to the length of the original message M .



There are many ways to solve this task. It will be instructive to look at the first few subtasks.

1 Subtask 1

For this subtask, recall that the original message M is just a sequence of two possible numbers, 0 or 1. Assuming that the original message is indexed from 0 to $N - 1$, one can instead choose to send the positions in the original message where all number 1's are located. This technique uses at most N integers to send the message.

2 Subtask 2

For subtask 2, since the numbers in the original message M could be from 0 to 255, they can be encoded using only 8 bits. However, this subtask allows one to use up to 16 bit per number in the encoded message E . Hence, you can encode the position of each number using the higher 8 bits (similar to the solution to subtask 1), and use the lower 8 bits to encode the actual number in the original message M , i.e.

$$E_i = 256 \cdot i + M_i$$

where E_i denotes the i^{th} number in the original message.

Note that the encoded numbers are also ordered, i.e. $E_i < E_{i+1}$, for $0 \leq i < N - 1$. Therefore, when we would like to decode the message from the shuffled array X , we can sort it to get E . Then, to obtain the original message from the sorted E should be pretty obvious.

This method also uses at most N parrots, which is satisfactory for this subtask.

3 Subtask 3

Thinking about the first two subtasks should get us familiar with the approach. There are many roads to take from here. However, to formalize the settings, we shall think of a general representation first.

Because we shall keep integers in the encoded array, a natural representation would be to use a sorted sequence. Note that this representation is very robust against permutation; provided that the encoded array E is sorted, by sorting a shuffled array X , we can easily recover the encoded array E , as in subtask 2.

We shall try to extend the approach in subtask 2. However, for subtasks 3–5, we do not have “extra” bits for keeping the position data. In these cases, we look at the original message as a sequence of binary digits (or bits), 0 and 1, by representing each number using only 8 bits. Thus, the original message of length N will have $8N$ bits.

This leads to a way to solve subtask 3. Note that the original message M contains at most $16 \times 8 = 128$ bits. For now, let B_i be the i^{th} bit of the original message, indexing from 0 to $8N - 1$. In this case, it is sufficient to take 7 bits to represent all positions of every single bit B_i , and use one last bit to represent the actual data. Hence, the encoding scheme looks like the following

$$E_i = 2 \cdot i + B_i$$

therefore, it takes 7 bits to represent possible positions and another bit to keep the data. That is, for each bit b_i , we shall encode it as $2 \cdot i + b_i$. The length of the encoded message for each test case of this subtask is $8N$ bytes.

4 Subtask 4

For this subtask, there are $32 \times 8 = 256$ bit positions; and it requires 8 bits just to represent the positions. Hence, there is no way to include the information of each bit of the original message in the encoding. If we recall the technique we use to solve subtask 1, we can improve the technique we used in subtask 3 by choosing to encode the positions of all 1 bits. This approach again sends at most $8N$ bytes.

5 Subtask 5

For this subtask, contestants are allowed to obtain some partial scores, depending on the ratio of the encoding message to the original message. In our point of view, to obtain *nearly perfect* scores like 98 is relatively much easier than to obtain the full points. We consider the last 2 points to be *the reward* to those who choose to implement more sophisticated method of encoding that achieve better results than our expectations.

We discuss many potential solutions that might achieve some partial scores.

5.1 A *ratio=12* approach

We are going to generalize the solution to the previous subtask case since the number of bits is more than 256. In this case, there are at most 512 bit positions.

To deal with this issue, we partition the message into 256 pairs of bits. We define the i^{th} bit pair P_i to be (M_{2i}, M_{2i+1}) . Again, the bit pair P should be indexed from 0 to $4N - 1$.

Note that there are 4 possible values for each bit pair, which can be represented with integers from 0 to 3. One possible method is to send the positions of the bit pair (which uses 8 bit to encode) in the encoding data, and then the actual bit pair data will be encoded as the *number of occurrences* of such positions.

Using this approach, for each pair of bits, we may have to send at most 3 bytes. Therefore, we send at most $\frac{3}{2} \cdot 8N = 12N$ bytes. Contestants will achieve 7 points on this subtask for implementing this method.

5.2 A *ratio=6.25* approach

With a simple observation, we can reduce the size of the encoded messages by roughly a factor of two. Note that the best case (in terms of encoding length) for the previous encoding method is when the original message only contains zero bits, and the worst case is when the message contains only one bits. We shall try to get the average of these two cases.

Consider two encoding scheme, called ONE and ZERO. For a pair of bits (b_{2i}, b_{2i+1}) , the ONE scheme sends $2 \cdot b_{2i} + b_{2i+1}$ copies of i , but the ZERO scheme sends $3 - (2 \cdot b_{2i} + b_{2i+1})$ copies. Note that both schemes work, provided that the decoder knows which of the schemes the encoder actually uses.

For each pair of bits, the sum of the number of encoded bytes used by both scheme is always $2 \cdot b_{2i} + b_{2i+1} + 3 - (2 \cdot b_{2i} + b_{2i+1}) = 3$ bytes. Considering all pairs, the sum is $3 \cdot \frac{8N}{2} = 12N$. Now, if we choose the scheme with the lower number of encoding data, we will need at most half of this number, i.e., at most $6N$ bytes.

There are many ways to signal the decoder which encoding scheme the encoder uses without too much penalty on the ratio. For example, we can add 4 copies of 255 iff the ZERO scheme is used. This will make the ratio goes up by $\frac{4}{N}$. However, since $N \geq 16$, the worst ratio is at most 6.25. Contestants will achieve 17 points on this subtask for implementing this method.

5.3 Better approaches

There are many other approaches that give better compression ratio.

5.3.1 One byte to 4 numbers

Consider the number of encoded messages of length *no greater than* 7, only consisting of numbers 0, 1, 2, and 3 (0, 0, 1, 1, 1, 1, 3 is one such message). To analyze this, it might be easier to consider an equivalent scheme: encoded messages of length *exactly* 7, only consisting of numbers 0, 1, 2, 3, and BLANK. There are $\binom{4+7}{4} = \binom{11}{4} = 330$ of them, which is enough to encode one byte of the original message.

Since the original message is at most 64 bytes long, it is possible to allocate 4 numbers per byte (0, 1, 2, and 3 for the first byte, for example), and therefore numbers from 0 to 255 are sufficient. This scheme yields a compression ratio of at most 7.

5.3.2 Optimal ratio

Theoretically, one can encode 64 bytes of data to the minimum of 261 bytes of encoded message without loss of information. There are $\binom{256+261}{256} = \binom{517}{256} \approx 1.47 \times 10^{154}$ different encoded messages using no more than 261 bytes. This is just more than $256^{64} \approx 1.34 \times 10^{154}$, the number of different 64-byte original messages. The compression ratio achieved is about 4.08. For smaller data, the ratio is even smaller.

This method yields a perfect score for this task. However, implementation of this scheme is comparatively difficult.